

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems **NAME:** K.SARANYA
COURSE CODE: CSA0501 **REG NO:**192424409

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Scenario-Based Task (High)

Assessment Tool Weightage: 35%

Dr

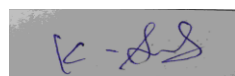
QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	5
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	5
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	5
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	5
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	5
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	5
7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	5
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	5

9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	5
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	5

Code of Conduct:

I K. SARANYA(192424409) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions	Makes reasonable decisions with	Makes weak decisions with limited	No clear decisions made or

		with strong rationale	some justification	justification	rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Table of Contents

- Q1. School Management System — Students, Class Teacher & Grade
- Q2. Hospital Appointment Tracking — Doctors with >10 Appointments
- Q3. E-Commerce — Products Never Ordered (Sub-queries)
- Q4. University Database — Relational Algebra (DBMS $\cap$ OS)
- Q5. Payroll — View for Below-Average Salary + Update
- Q6. Banking — JOIN across Customer, Account, Transaction, Branch
- Q7. Retail — GROUP BY / HAVING Daily Sales Summary
- Q8. Library — Tuple Relational Calculus (All-Genre Borrowers)
- Q9. Inventory — CHECK and FOREIGN KEY Constraints
- Q10. HR System — Schema + Materialized Views

Question 1: School Management System

Scenario: Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.

1. Understanding the Scenario

A School Management System must track which student belongs to which class, which grade that class represents, and which teacher is responsible for the class as its class teacher. This naturally leads to three entities: STUDENT, CLASS, and TEACHER, connected through foreign keys.

2. Relational Schema

- STUDENT (StudentID [PK], Name, ClassID [FK $\rightarrow$ CLASS], DOB)
- CLASS (ClassID [PK], ClassName, Grade, TeacherID [FK $\rightarrow$ TEACHER])
- TEACHER (TeacherID [PK], Name, Subject)

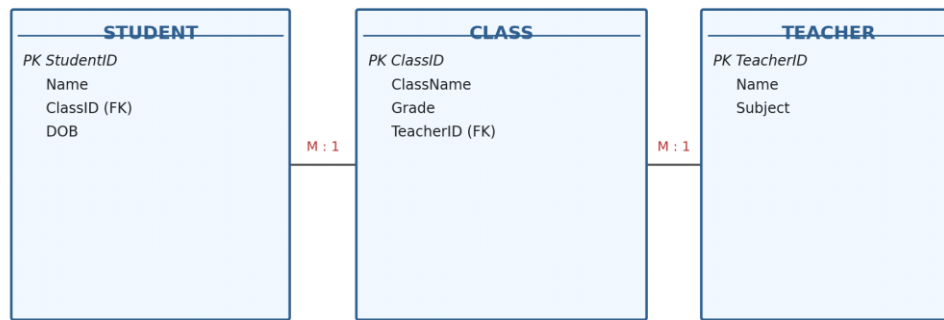


Figure 1.1 — ER Diagram: School Management System

3. SQL — Table Creation

```
CREATE TABLE Teacher (  
    TeacherID INT PRIMARY KEY,  
    Name     VARCHAR(50),  
    Subject  VARCHAR(50)  
);  
  
CREATE TABLE Class (  
    ClassID   INT PRIMARY KEY,  
    ClassName VARCHAR(20),  
    Grade     VARCHAR(10),  
    TeacherID INT,  
    FOREIGN KEY (TeacherID) REFERENCES Teacher(TeacherID)  
);  
  
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name      VARCHAR(50),  
    ClassID   INT,  
    DOB       DATE,  
    FOREIGN KEY (ClassID) REFERENCES Class(ClassID)  
);
```

4. SQL — Query: Students with Class Teacher and Grade

```
SELECT s.StudentID,  
       s.Name      AS StudentName,  
       c.ClassName,  
       c.Grade,  
       t.Name      AS ClassTeacher  
FROM   Student s
```

```
JOIN Class c ON s.ClassID = c.ClassID
JOIN Teacher t ON c.TeacherID = t.TeacherID
ORDER BY c.Grade, s.Name;
```

5. Explanation

The query performs two INNER JOINS. Student is joined to Class on ClassID to obtain the class name and grade, and Class is joined to Teacher on TeacherID to obtain the name of the class teacher responsible for that class. The result is one row per student showing the student's name, class, grade, and class teacher, ordered by grade for readability.

Question 2: Hospital Appointment Tracking

Scenario: A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.

1. Understanding the Scenario

The hospital needs to record doctors, patients, and the appointments that link them together, including the date of each appointment so that monthly activity can be measured per doctor.

2. Relational Schema

- DOCTOR (DoctorID [PK], Name, Specialization)
- PATIENT (PatientID [PK], Name, Phone)
- APPOINTMENT (ApptID [PK], DoctorID [FK → DOCTOR], PatientID [FK → PATIENT], ApptDate)

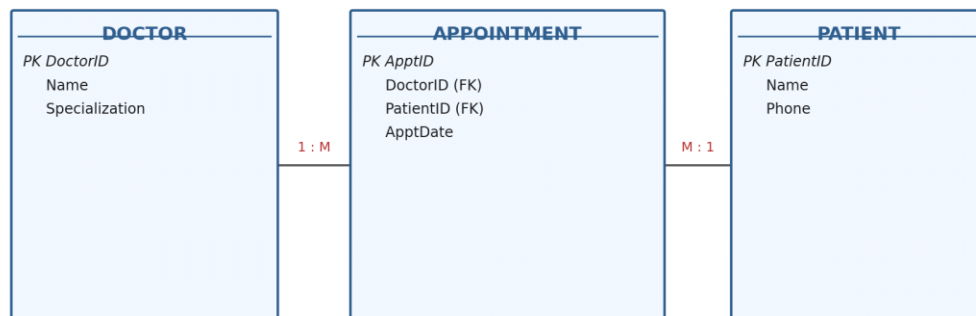


Figure 2.1 — ER Diagram: Hospital Appointment System

3. SQL — Table Creation

```
CREATE TABLE Doctor (
```

```

DoctorID    INT PRIMARY KEY,
Name        VARCHAR(50),
Specialization VARCHAR(50)
);

CREATE TABLE Patient (
    PatientID INT PRIMARY KEY,
    Name      VARCHAR(50),
    Phone     VARCHAR(15)
);

CREATE TABLE Appointment (
    ApptID    INT PRIMARY KEY,
    DoctorID  INT,
    PatientID INT,
    ApptDate  DATE,
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
);

```

4. SQL — Doctors with More Than 10 Appointments This Month

```

SELECT d.DoctorID,
       d.Name      AS DoctorName,
       COUNT(a.ApptID) AS TotalAppointments
FROM   Doctor d
JOIN   Appointment a ON d.DoctorID = a.DoctorID
WHERE  MONTH(a.ApptDate) = MONTH(CURDATE())
      AND YEAR(a.ApptDate) = YEAR(CURDATE())
GROUP BY d.DoctorID, d.Name
HAVING COUNT(a.ApptID) > 10;

```

5. Explanation

The WHERE clause filters Appointment rows down to the current calendar month and year. Grouping by doctor and applying COUNT() gives the number of appointments handled by each doctor within that period. The HAVING clause then keeps only those doctors whose appointment count exceeds 10 — HAVING is required here (rather than WHERE) because the filter is applied to an aggregated value.

Question 3: E-Commerce — Products Never Ordered

Scenario: In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.

1. Understanding the Scenario

An online store lists products, and customers place orders that reference one or more products through an order-details line-item table. A product that never appears in any order line item is effectively unsold and should be reported for inventory review.

2. Relational Schema

- PRODUCT (ProductID [PK], PName, Price)
- ORDERS (OrderID [PK], CustID, OrderDate)
- ORDER_DETAILS (OrderID [FK], ProductID [FK], Qty) — composite key (OrderID, ProductID)

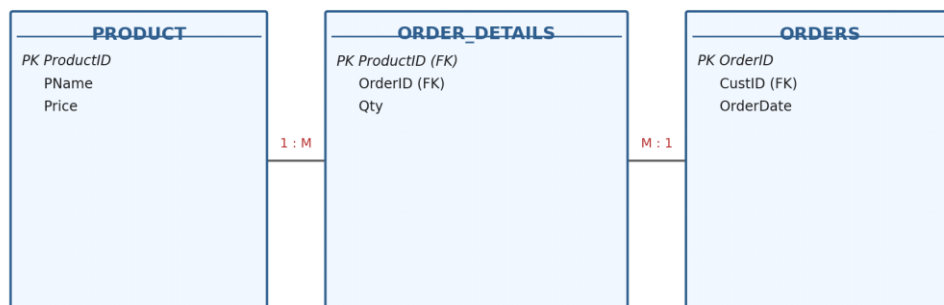


Figure 3.1 — ER Diagram: E-Commerce Order System

3. SQL — Sub-query Using NOT IN

```
SELECT ProductID, PName, Price
FROM Product
WHERE ProductID NOT IN (
    SELECT ProductID
    FROM Order_Details
);
```

4. SQL — Equivalent Sub-query Using NOT EXISTS (Safer)

```
SELECT p.ProductID, p.PName, p.Price
FROM Product p
WHERE NOT EXISTS (
    SELECT 1
    FROM Order_Details od
    WHERE od.ProductID = p.ProductID
);
```

5. Explanation

Both queries return products whose ProductID never appears in Order_Details. NOT IN is intuitive but silently returns zero rows if the sub-query produces even one NULL value, so the NOT EXISTS form is recommended in production because it is NULL-safe and typically performs at least as well since the optimiser can use a semi-join / anti-join strategy.

Question 4: University Database — Relational Algebra

Scenario: For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.

1. Understanding the Scenario

A student can enroll in many courses, and a course can have many students, so ENROLLMENT is a many-to-many bridge between STUDENT and COURSE. We need the set of students who appear in the enrollment set for both named courses simultaneously — a set intersection problem.

2. Relational Schema

- STUDENT (StudentID, Name)
- COURSE (CourseID, CourseName)
- ENROLLMENT (StudentID, CourseID)

3. Relational Algebra Expression

Let $E = \text{ENROLLMENT} \bowtie \text{COURSE}$ (natural join on CourseID)

$R1 = \pi_{\text{StudentID}} (\sigma_{\text{CourseName} = \text{'DBMS'}} (E))$

$R2 = \pi_{\text{StudentID}} (\sigma_{\text{CourseName} = \text{'OS'}} (E))$

Result = $R1 \cap R2$

4. Full Expression in a Single Line

$\pi_{\text{StudentID}} (\sigma_{\text{CourseName} = \text{'DBMS'}} (\text{ENROLLMENT} \bowtie \text{COURSE}))$

$\cap$

$\pi_{\text{StudentID}} (\sigma_{\text{CourseName} = \text{'OS'}} (\text{ENROLLMENT} \bowtie \text{COURSE}))$

5. Equivalent SQL (for reference)

```
SELECT e1.StudentID
FROM Enrollment e1 JOIN Course c1 ON e1.CourseID = c1.CourseID
WHERE c1.CourseName = 'DBMS'
```



```
INTERSECT
SELECT e2.StudentID
FROM Enrollment e2 JOIN Course c2 ON e2.CourseID = c2.CourseID
WHERE c2.CourseName = 'OS';
```

6. Explanation

σ (selection) first restricts the joined Enrollment–Course rows to the DBMS course and, separately, to the OS course. π (projection) reduces each restricted set down to just the StudentID column. The set-intersection operator $\cap$ then keeps only the student identifiers that occur in both projected sets — exactly the students enrolled in both courses.

Question 5: Payroll — View and Update

Scenario: Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.

1. Understanding the Scenario

Every employee belongs to a department and earns a salary. To identify under-paid staff relative to peers, we compare each employee's salary against the average salary of their own department (a correlated sub-query), then encapsulate that logic in a view so it can be reused, and finally apply a 10% raise to exactly those employees.

2. Relational Schema

- DEPARTMENT (DeptID [PK], DeptName)
- EMPLOYEE (EmpID [PK], Name, DeptID [FK], Salary)

3. SQL — Create the View

```
CREATE VIEW BelowAvgSalary AS
SELECT e.EmpID, e.Name, e.DeptID, e.Salary
FROM Employee e
WHERE e.Salary < (
    SELECT AVG(e2.Salary)
    FROM Employee e2
    WHERE e2.DeptID = e.DeptID
);
```

4. SQL — Query the View

```
SELECT * FROM BelowAvgSalary ORDER BY DeptID;
```

5. SQL — Update Salaries of Employees in the View by 10%

```
UPDATE Employee
SET Salary = Salary * 1.10
WHERE EmpID IN (SELECT EmpID FROM BelowAvgSalary);
```

6. Explanation

The view's WHERE clause is a correlated sub-query: for every candidate employee e, it recomputes the average salary of e's own department ($e2.DeptID = e.DeptID$) and compares e's salary against that average. Because the view is derived from a single base table with a correlated condition, most database engines treat it as updatable; even so, the UPDATE statement is written directly against the base Employee table using the view purely to select the target EmpID values, which is the safest and most portable approach across MySQL, PostgreSQL and Oracle.

Question 6: Banking — JOIN Across Multiple Tables

Scenario: In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.

1. Understanding the Scenario

A bank customer holds one or more accounts, each account belongs to a branch, and each account accumulates a history of transactions. A complete customer statement therefore requires joining four tables together.

2. Relational Schema

- CUSTOMER (CustID [PK], Name, Address)
- BRANCH (BranchID [PK], BranchName, City)
- ACCOUNT (AccNo [PK], CustID [FK], BranchID [FK], Balance)
- TRANSACTION (TxnID [PK], AccNo [FK], Type, Amount, TxnDate)

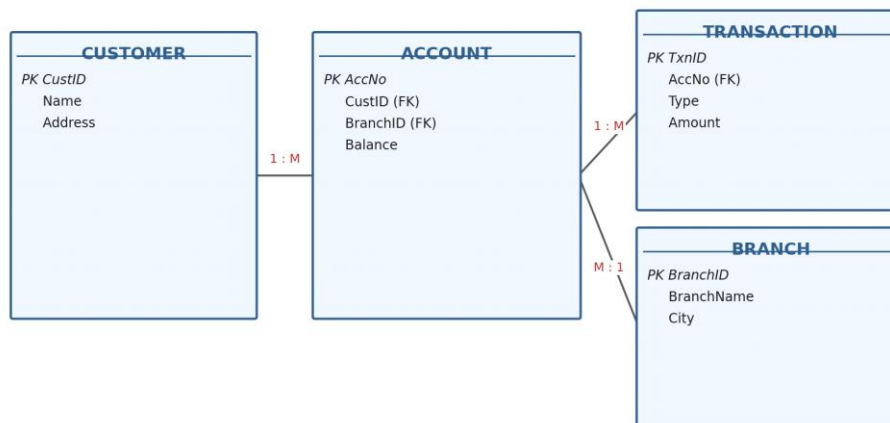


Figure 6.1 — ER Diagram: Banking System

3. SQL — Multi-table JOIN

```
SELECT c.CustID,  
       c.Name      AS CustomerName,  
       a.AccNo,  
       a.Balance,  
       b.BranchName,  
       b.City,  
       t.TxnID,  
       t.Type      AS TxnType,  
       t.Amount,  
       t.TxnDate  
FROM   Customer c  
JOIN   Account  a ON c.CustID = a.CustID  
JOIN   Branch   b ON a.BranchID = b.BranchID  
LEFT JOIN Transaction t ON a.AccNo = t.AccNo  
ORDER BY c.CustID, t.TxnDate;
```

4. Explanation

Customer, Account and Branch are combined with INNER JOINS because every account must belong to exactly one customer and one branch. Transaction is attached with a LEFT JOIN instead, so that accounts which have not yet made any transaction still appear in the result (with NULL transaction columns) rather than being silently dropped, which an INNER JOIN would do.

Question 7: Retail — Daily Sales Summary

Scenario: A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.

1. Understanding the Scenario

Every sale is recorded against a product category. Management wants a same-day report of total revenue per category, restricted to categories that have crossed the Rs. 10,000 threshold for the day.

2. Relational Schema

- SALES (SaleID [PK], ProductID, Category, Amount, SaleDate)

3. SQL — Daily Category-wise Sales Exceeding Rs. 10,000

```

SELECT  Category,
        SaleDate,
        SUM(Amount) AS TotalSales
FROM    Sales
WHERE   SaleDate = CURDATE()
GROUP BY Category, SaleDate
HAVING  SUM(Amount) > 10000
ORDER BY TotalSales DESC;

```

4. Explanation

GROUP BY Category collapses all of today's sale rows into one group per category. SUM(Amount) then computes the total revenue for each group. HAVING filters the grouped results, keeping only categories whose group total is above Rs. 10,000 — a row-level WHERE clause could not do this because Amount has not yet been aggregated at the point WHERE is evaluated.

Question 8: Library System — Tuple Relational Calculus

Scenario: For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.

1. Understanding the Scenario

This is a relational-division problem: we want members whose set of borrowed-book genres covers every genre that exists in the library — i.e. for every genre g in the library, the member has borrowed at least one book of genre g .

2. Relational Schema

- MEMBER (MemberID [PK], Name)
- BOOK (BookID [PK], Title, Genre)
- BORROW (MemberID [FK], BookID [FK], BorrowDate)

3. Tuple Relational Calculus (TRC) Expression

$$\{ t \mid \exists m \in \text{MEMBER} (t.\text{MemberID} = m.\text{MemberID} \wedge t.\text{Name} = m.\text{Name}) \\
 \wedge \\
 \forall g \in \text{BOOK} (\\
 \quad \exists bk \in \text{BOOK} (bk.\text{Genre} = g.\text{Genre} \\
 \quad \wedge \exists br \in \text{BORROW} (br.\text{BookID} = bk.\text{BookID} \\
 \quad \quad \wedge br.\text{MemberID} = m.\text{MemberID})) \\
)$$

```
}
```

4. Reading the Expression

The outer existential quantifier binds *m* to a member. The universal quantifier ranges over every book *g* in the BOOK relation purely to enumerate every distinct genre. For each such genre, the inner existential clause demands that there exist some book *bk* of that same genre which the member *m* has actually borrowed (recorded in BORROW). If this inner condition holds for every genre without exception, tuple *t* — the member — belongs to the result.

5. Equivalent SQL (Division via Double NOT EXISTS)

```
SELECT m.MemberID, m.Name
FROM   Member m
WHERE  NOT EXISTS (
    SELECT DISTINCT bk.Genre
    FROM   Book bk
    WHERE  NOT EXISTS (
        SELECT 1
        FROM   Borrow br
        JOIN   Book b2 ON br.BookID = b2.BookID
        WHERE  br.MemberID = m.MemberID
              AND b2.Genre = bk.Genre
    )
);
```

6. Explanation

The SQL mirrors classic relational division: the outer NOT EXISTS discards any member for whom there remains at least one genre (inner query) that has no matching borrowed book of that genre. Only members with zero such 'missing genres' survive — precisely the members who have borrowed from every genre.

Question 9: Inventory Management — CHECK and FOREIGN KEY

Scenario: Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.

1. Understanding the Scenario

An inventory system must guarantee that stock quantities never go negative (a business rule enforced with CHECK) and that every product is linked to a valid, existing supplier (referential integrity enforced with FOREIGN KEY).

2. Relational Schema

- SUPPLIER (SupplierID [PK], SName, Contact)
- PRODUCT (ProductID [PK], PName, Quantity, SupplierID [FK → SUPPLIER])

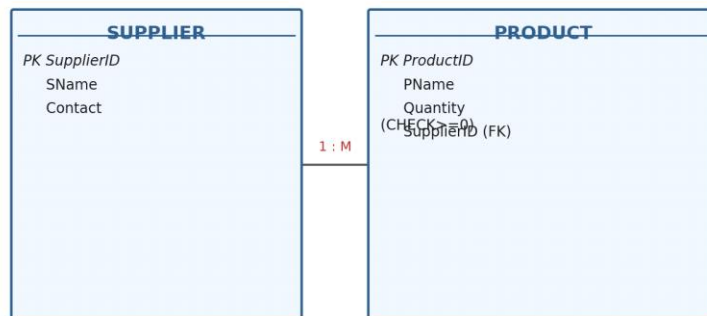


Figure 9.1 — ER Diagram: Inventory System

3. SQL — Table Definitions with Constraints

```
CREATE TABLE Supplier (  
    SupplierID INT PRIMARY KEY,  
    SName    VARCHAR(50),  
    Contact  VARCHAR(15)  
);  
  
CREATE TABLE Product (  
    ProductID INT PRIMARY KEY,  
    PName    VARCHAR(50),  
    Quantity  INT NOT NULL CHECK (Quantity >= 0),  
    SupplierID INT,  
    CONSTRAINT fk_supplier  
        FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)  
        ON DELETE RESTRICT  
        ON UPDATE CASCADE  
);
```

4. Demonstrating CHECK Enforcement

```
-- Rejected: violates CHECK (Quantity >= 0)  
INSERT INTO Product (ProductID, PName, Quantity, SupplierID)  
VALUES (101, 'Steel Rod', -5, 1);  
-- Error: Check constraint 'Quantity >= 0' is violated.  
  
-- Accepted  
INSERT INTO Product (ProductID, PName, Quantity, SupplierID)
```

```
VALUES (101, 'Steel Rod', 50, 1);
```

5. Demonstrating FOREIGN KEY Enforcement

```
-- Rejected: SupplierID 99 does not exist in Supplier
INSERT INTO Product (ProductID, PName, Quantity, SupplierID)
VALUES (102, 'Copper Wire', 20, 99);
-- Error: Foreign key constraint 'fk_supplier' fails.

-- Rejected: cannot delete a supplier still referenced by a product
DELETE FROM Supplier WHERE SupplierID = 1;
-- Error: Cannot delete parent row: a foreign key constraint fails
-- (fk_supplier, ON DELETE RESTRICT).
```

6. Explanation

The CHECK constraint blocks any INSERT or UPDATE that would leave Quantity negative, protecting the business rule at the database level regardless of which application writes to the table. The FOREIGN KEY constraint blocks Product rows that reference a non-existent SupplierID, and ON DELETE RESTRICT further prevents a supplier from being removed while products still depend on it, preserving referential integrity.

Question 10: HR System — Schema and Materialized Views

Scenario: Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.

1. Understanding the Scenario

An HR system tracks employees, their daily attendance records, and periodic performance scores. Because monthly summary reports are read frequently but the underlying data changes gradually, materialized views are ideal: they pre-compute and physically store the aggregated result so reports are fast, refreshing on a schedule rather than recalculating on every read.

2. Relational Schema

- EMPLOYEE (EmpID [PK], Name, DeptID, DOJ)
- ATTENDANCE (AttID [PK], EmpID [FK], Date, Status) — Status ∈ {Present, Absent, Leave}
- PERFORMANCE (PerfID [PK], EmpID [FK], Month, Score)

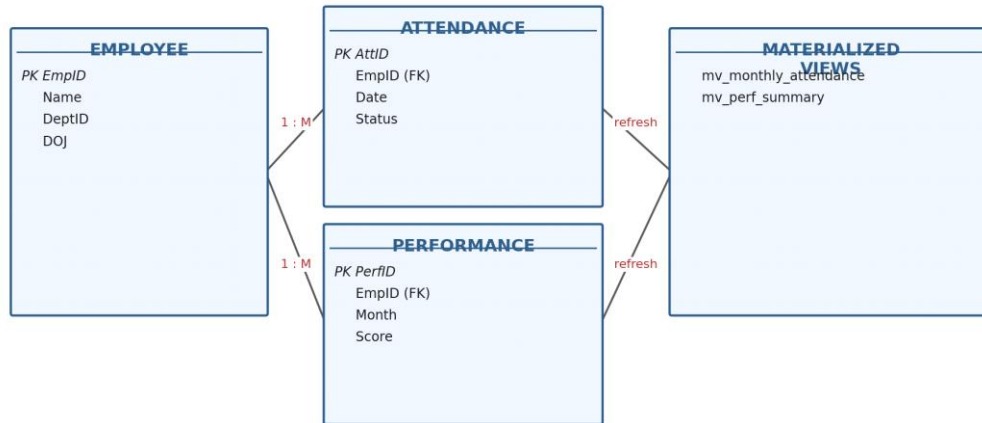


Figure 10.1 — ER Diagram: HR System with Materialized Views

3. SQL — Base Table Creation

```

CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(50),
    DeptID INT,
    DOJ DATE
);

CREATE TABLE Attendance (
    AttID INT PRIMARY KEY,
    EmpID INT REFERENCES Employee(EmpID),
    Date DATE,
    Status VARCHAR(10)
);

CREATE TABLE Performance (
    PerfID INT PRIMARY KEY,
    EmpID INT REFERENCES Employee(EmpID),
    Month DATE,
    Score DECIMAL(4,2)
);
  
```

4. Materialized View — Monthly Attendance Summary (PostgreSQL / Oracle syntax)

```

CREATE MATERIALIZED VIEW mv_monthly_attendance AS
SELECT e.EmpID,
       e.Name,
       DATE_TRUNC('month', a.Date) AS AttMonth,
       COUNT(*) FILTER (WHERE a.Status = 'Present') AS DaysPresent,
       COUNT(*) FILTER (WHERE a.Status = 'Absent') AS DaysAbsent,
  
```



```

COUNT(*)                AS TotalDays
FROM   Employee e
JOIN   Attendance a ON e.EmpID = a.EmpID
GROUP BY e.EmpID, e.Name, DATE_TRUNC('month', a.Date);

-- Refresh on a schedule (e.g. nightly job)
REFRESH MATERIALIZED VIEW mv_monthly_attendance;

```

5. Materialized View — Monthly Performance Summary

```

CREATE MATERIALIZED VIEW mv_perf_summary AS
SELECT e.EmpID,
       e.Name,
       p.Month,
       AVG(p.Score) AS AvgScore
FROM   Employee e
JOIN   Performance p ON e.EmpID = p.EmpID
GROUP BY e.EmpID, e.Name, p.Month;

REFRESH MATERIALIZED VIEW mv_perf_summary;

```

6. MySQL Equivalent (No Native Materialized View)

```

-- MySQL lacks CREATE MATERIALIZED VIEW; emulate with a summary
-- table refreshed by a scheduled event.
CREATE TABLE mv_monthly_attendance AS
SELECT e.EmpID, e.Name,
       DATE_FORMAT(a.Date, '%Y-%m-01') AS AttMonth,
       SUM(a.Status='Present') AS DaysPresent,
       SUM(a.Status='Absent') AS DaysAbsent
FROM   Employee e JOIN Attendance a ON e.EmpID = a.EmpID
GROUP BY e.EmpID, e.Name, AttMonth;

CREATE EVENT refresh_attendance_mv
ON SCHEDULE EVERY 1 DAY
DO
  REPLACE INTO mv_monthly_attendance
  SELECT ... /* same SELECT as above */;

```

7. Explanation

Both materialized views pre-aggregate data that would otherwise require repeated JOINS and GROUP BY operations on every report request. `mv_monthly_attendance` summarizes present/absent day counts per employee per month, and `mv_perf_summary` summarizes average performance score per employee per month. Because materialized views store their result physically, they must be explicitly refreshed (`REFRESH MATERIALIZED VIEW`, or a

scheduled job in MySQL) after the underlying Attendance or Performance data changes, trading a small staleness window for significantly faster report reads.